

The Role of AI and LLMs in Developing Parallel Code

1st Dipika Bogati
Computer Science
Bowling Green State University
Bowling Green, USA
dbogati@bgsu.edu

2nd Huy Le Vuong
Computer Science
Bowling Green State University
Bowling Green, USA
hvuong@bgsu.edu

3rd Radhika Srivastava
Computer Science
Bowling Green State University
Bowling Green, USA
rsriva@bgsu.edu

Abstract—

Index Terms—component, formatting, style, styling, insert

I. INTRODUCTION

II. RELATED WORK

The paper [5] show how AI can speed up the process of finding efficient GPU code. Traditional tools like TVM/Ansor test thousands of different ways to run the same program on a GPU, which can take 15-30 minutes for a single task. This research makes the process faster by introducing a simpler model that focuses on seven key hardware factors, such as data movement and thread utilization together with a hill-climbing style search strategy that steadily moves toward better versions instead of relying on random guesses. With these improvements, the system can generate superior quality code in only 1-2 minutes.

[11] present a survey of how AI techniques can address the long-standing challenges of parallel programming, such as dividing tasks across processors, managing memory, and ensuring correct synchronization. The paper reviews three main AI approaches. Genetic algorithms are described as effective for exploring large and complex solution spaces in areas such as task distribution, synchronization strategies, and memory allocation, although they can suffer from slow convergence and variability across runs. Reinforcement learning is seen as a promising way to dynamically manage resources and threads where agents can learn by trial and error how to balance workloads, create or terminate threads, and adapt to changing system conditions. Neural networks are positioned as especially useful for automatic parallel code generation, and performance prediction.

New techniques like transfer learning and neurosymbolic methods make AI tools even more powerful. Transfer learning helps models adapt to different types of tasks without starting from scratch, while neurosymbolic methods improve accuracy and make results easier to explain, especially in tasks such as debugging and error detection.

The paper also points out several practical ways AI can help in parallel programming. AI can automatically turn sequential programs into parallel programs that run efficiently on specific hardware. It can also manage resources on the fly, deciding how to assign tasks and memory to different cores depending

on the workload which makes the system faster and more efficient.

The paper [6] assess the ability of LLMs to generate parallel code using OpenMP, focusing on chatGPT 3.5 and GitHub Copilot. They tested both tools on nine different applications, ranging from simple numerical tasks to more complex simulations, by asking them to transform sequential programs into parallel versions. The results show that both models can produce working parallel code, but with notable differences in quality. chatGPT was able to generate correct solutions in simpler cases, though it often required multiple rounds of synchronization. GitHub Copilot, on the other hand, provided more reliable output, needed less user intervention, and often achieved performance closer to manually optimized code. However, they are not yet a substitute for human expertise in performance-critical applications.

LLMCompiler is a system proposed by Kim et al. (2024) that allows for parallel function calls instead of sequential, improving the way LLMs interact with external tools. Current methods, like the ReAct process tool, invoke the process one step at a time, increasing latency, cost and the possibility of errors such as incomplete calls. LLM compiler addresses these limitations through a compiler inspired design that includes a planner to map tasks and their dependencies, a dispatcher to manage task execution as inputs become available, and an executor that runs tasks in parallel [4].

To generate the high performance parallel code generation, authors have created a new architecture which has four Parallel Code Generation Agents: Memory, Planning, Tools and Action which are connected to the main Agent [2]. Memory agent unifies the internal knowledge generated from the Fine-Tuning and external knowledge generated from Retrieval-Augmented Generation (RAG). Planning agent includes the prompting task which consists of both single-stage prompt and multi-stage prompt. Tools is used in the verification pipeline where the generated C++ code is compiled using GCC, the abstract Syntax Tree (AST) is employed in nm tool. Action defines the agent's capabilities for state transitions (edits, evaluations, resets) during the process of code optimization. They did the data augmentation on the publicly available **PIE** dataset using open-source model and the APIs to synthesize the code. To actually increase the performance of the parallel

code, planning-oriented prompting (prompt strategies: parallel, serial, any and multi-stage), code verification and Retrieval Augmented Editing are implemented. The experimental results shows that x6.06 speedup is achieved in Qwen2.5-Coder-7B-Instruct and x5.13 speedup is achieved in Qwen2.5-Coder-14B-Instruct LLM models [2].

Research was performed to design LLM on solving the High Performance Computing (HPC) [7]. The HPC code is generated from the GitHub repositories with the rating grater than 3. The HPC Code are filtered using deduplication, and files over 1MB and under 15 tokens are also removed. Using this dataset, three language models: GPT-2 with 1.5B parameters, GPT-Neo with 2.7B parameters, and PolyCoder with 2.7B parameters are fine-tuned. Using the fine-tuned models, set of HPC functions are generated(text generation) and performance and correctness of the generated code are evaluated. In addition, models' ability to label the for loops are also evaluated in OpenMP pragmas for which **for loops** are taken as input and OpenMP pragma are generated as the output while fine-tuning. The experiment results shows that the generated code speedup is always > 1 which means the LLM generated HPC code is correct and parallelized [7].

To improve Parallel Program Performance, high-quality mapper is required which assigns tasks to processors and data to memory properly which requires deep knowledge about hardware, low-level system APIs and applications which is difficult for the domain professionals. So, the paper [10] proposed a new framework to automate the generation and optimization of mapper code using LLMs using Agent-System Interface. One of the core Agent-System Interface is Domain-Specific Language which has declarative design (allowing users to declare what to achieve instead of long C++ code). Then AutoGuide is used to take in the raw logs and error messages from the mapper and translates them into the clear guidance. Finally, generative optimization loop is implemented which takes two inputs: application metadata and server specifications which define the search space explored by agents in the process of optimization. Hence, by utilizing the given inputs, the agent produces the mapper code that is executed on the application code in server. Authors have compared the results generated from their approach with OpenTuner and human expert and concluded that their method achieve 3.8 x faster in 10 iterations compared to 1000 iterations in OpenTuner and 1.34 x higher speedup than human expert written mappers [10].

There were several works on building/fine-tuning a domain-specific language models for OpenMP pragma generation mostly using the HPCorpus dataset. Those domain-specific LLMs demonstrates superior performance over existing language models in HPC contexts (Chen et al. 2024 [1]) and (Schneider et al. 2024 [9])

Chen et al. (2024) introduce OMPGPT, a novel domain-specific language model designed specifically for OpenMP pragma generation in high-performance computing (HPC). While general code-based large language models (LLMs) like StarCoder and CodeLlama are useful for many programming

tasks, HPC has unique requirements that benefit from a smaller, specialized model such as OMPGPT. OMPGPT used 'Chain-of-OMP', an innovative prompt engineering strategy to further enhance the effectiveness of the LLM. OMPGPT outperforms existing large language models specialized in OpenMP tasks. Additionally, OMPGPT is notably small in size, making it more suitable for hardware constraints typical in HPC environments [1].

Since existing static tools have limited effectiveness, and general-purpose language models like GPT-3.5 perform poorly on MPI code generation compared to standard programming tasks, Schneider et al. 2024 show that domain-specific models outperform larger, general models. They introduce MPIrigen, a model fine-tuned on a dataset containing MPI-related C/C++ code (HPCorpus MPI). MPIrigen significantly outperforms GPT-3.5 in generating accurate MPI function calls which proves domain-specific fine-tuned LLMs is more suitable for parallel computing code generation [9].

Kadosh et al., 2024 introduce OMPar, an AI-driven tool that automates the parallelization of C/C++ code using OpenMP pragmas. OMPar leverages Large Language Models (LLMs) through two main components: OMPify, which evaluates the potential for loop parallelization, and MonoCoder-OMP, a fine-tuned model that generates accurate OpenMP pragmas. Results show that OMPar outperforms existing methods in identifying parallelizable loops and generating efficient pragmas. Additionally, OMPar can handle partial or incomplete codebases and continuously improve by learning from new code patterns [3].

Nichols et al., 2024 use LLMs for automating complex development and analysis tasks in HPC. The authors introduce a new dataset of HPC and scientific codes and use it to fine-tune several pre-trained LLMs called HPC-Coder, a model specifically fine-tuned on parallel codes. Experiments show that HPC-Coder can auto-complete HPC functions (where generic models cannot), add OpenMP pragmas to loops, and model performance changes in scientific application repositories and programming competition solutions [8].

REFERENCES

- [1] Le Chen, Arijit Bhattacharjee, Nesreen K. Ahmed, Niranjan Hasabnis, Gal Oren, Vy A. Vo, and Ali Jannesari. Ompgpt: A generative pre-trained transformer model for openmp. In *European Conference on Parallel Processing*, 2024.
- [2] Xiaowen Huang, Xu Zhang, Lvfang Tao, Renjie Mao, Nan Zhou, Wenxi Zhu, Minwen Deng, Jintao Meng, Yanjie Wei, Amelie Chi Zhou, et al. Paracoder: Parallel code generation with large language model. In *Proceedings of the 1st FastCode Programming Challenge*, pages 1–7. 2025.
- [3] Tal Kadosh, Niranjan Hasabnis, Prema Soundararajan, Vy A. Vo, Mihai Capota, Nesreen K. Ahmed, Yuval Pinter, and Gal Oren. Ompar: Automatic parallelization with ai-driven source-to-source compilation. *ArXiv*, abs/2409.14771, 2024.
- [4] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling, 2024.
- [5] Chendi Li, Yufan Xu, Sina Mahdipour Saravani, and Ponnuswamy Sadayappan. Accelerated auto-tuning of gpu kernels for tensor computations. In *Proceedings of the 38th ACM International Conference on Supercomputing*, ICS '24, page 549–561, New York, NY, USA, 2024. Association for Computing Machinery.

- [6] Marko Mišić and Matija Dodović. An assessment of large language models for openmp-based code parallelization: a user perspective. *Journal of Big Data*, 11, 11 2024.
- [7] Daniel Nichols, Aniruddha Marathe, Harshitha Menon, Todd Gamblin, and Abhinav Bhatele. Modeling parallel programs using large language models. *arXiv preprint arXiv:2306.17281*, 2023.
- [8] Daniel Nichols, Aniruddha Marathe, Harshitha Menon, Todd Gamblin, and Abhinav Bhatele. Hpc-coder: Modeling parallel programs using large language models. In *ISC High Performance 2024 Research Paper Proceedings (39th International Conference)*, pages 1–12, 2024.
- [9] Nadav Schneider, Niranjana Hasabnis, Vy A. Vo, Tal Kadosh, Neva Krien, Mihai Capotua, Abdul Wasay, Guy Tamir, Ted Willke, Nesreen K. Ahmed, Yuval Pinter, Timothy Mattson, and Gal Oren. Mpirigen: Mpi code generation through domain-specific language models. *Proceedings of the 2024 Workshop on AI For Systems*, 2024.
- [10] Anjiang Wei, Allen Nie, Thiago SFX Teixeira, Rohan Yadav, Wonchan Lee, Ke Wang, and Alex Aiken. Improving parallel program performance with llm optimizers via agent-system interfaces. *arXiv preprint arXiv:2410.15625*, 2024.
- [11] Rimma Zaripova, Adam Mentsiev, and Olga Kovrizhnykh. Advancing parallel programming integrating artificial intelligence for enhanced efficiency and automation. *E3S Web of Conferences*, 460:1–9, 12 2023.